Interpolation & Extrapolation: Summary

The main ideas on a few slides

Sharareh Sayyad

Washington State University

October 16, 2026

The task

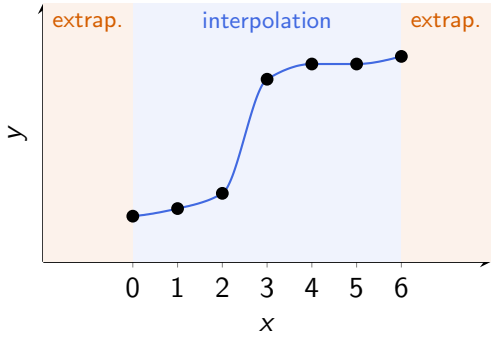
Given data (x_i, y_i)

Find a function p that passes through every point:

$$p(x_i) = y_i$$

- **Interpolation:** evaluate p *between* data points.
- **Extrapolation:** evaluate p *outside* the data range.

Infinitely many curves pass through the points.
Each method = one rule for what happens between them.



Three ideas in 1-D

Linear interpolation

1. Connect the dots

$$p = (1 - t)\, y_i + t\, y_{i+1}$$

- local, never overshoots
- corners at the nodes
- error $\leq \frac{h^2}{8} \max |f''|$
  (halve $h \Rightarrow$ error $\div 4$)

Polynomial interpolation

2. One global polynomial

$n + 1$ points $\rightarrow$ degree n

- perfectly smooth
- every point affects everything
- wiggles between the points

Piecewise polynomial interpolation

3. Piecewise cubics

On each interval a cubic with

- 2 values fixed: y_i, y_{i+1}
- 2 slopes d_i, d_{i+1} **chosen**

Smooth, but each piece stays local.

Spline, PCHIP, Akima, Makima differ **only** in how they choose the slopes d_i

Choosing the slopes: spline vs. PCHIP

Cubic spline

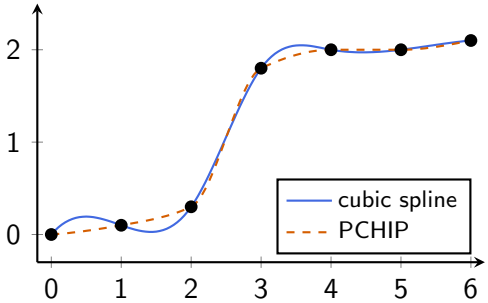
Require p'' continuous $\Rightarrow$

$$d_{i-1} + 4d_i + d_{i+1} = 3(\delta_{i-1} + \delta_i)$$

global system; smoothest, most accurate on smooth data, can overshoot.

PCHIP

$d_i = 0$ if the data turns or is flat, else a harmonic mean of δ_{i-1}, δ_i.
local, shape preserving: no new bumps.
(Akima / Makima: local cousins.)



Trade-off: **smoothness** vs. **locality**.

Which one, and how in SciPy

Rule of thumb

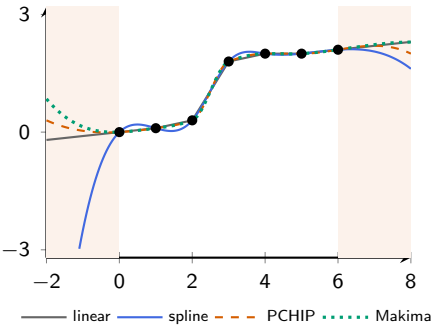
- smooth data, want accuracy ⇒ **cubic spline**
- must keep shape (no overshoot, monotone, positive) ⇒ **PCHIP**
- smooth-looking but local, mild overshoot only ⇒ **Makima**
  (Akima with the flat-data overshoot fixed)
- quick and safe ⇒ **linear**

method	call
linear	`np.interp(x0, x, y)`
	`make_interp_spline(x, y, k=1)`
spline	`CubicSpline(x, y)`
PCHIP	`PchipInterpolator(x, y)`
Makima	`Akima1DInterpolator(x, y, method="makima")`

All return an object p; evaluate with `p(x0)`.

Extrapolation: the big idea and how safe each method is

Outside the data there is **no information**: each method just continues its last piece.



Inside $[0, 6]$ they agree; outside they fan out.

method	outside	$x = -2$	safety
`np.interp`	holds end value	0.0	bounded, flat
linear (k=1)	last line	−0.2	slow drift
cubic spline	last cubic	−9.9	can blow up
PCHIP	last cubic	0.3	shape lost
Akima / Makima	`nan` (default)	`nan`	warns you

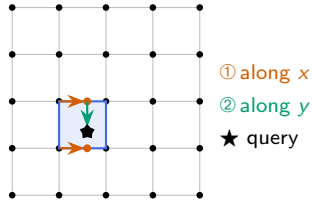
PCHIP is monotone inside, but not outside: 2.2 at x=7, 2.0 at x=8.

No method is safe in general

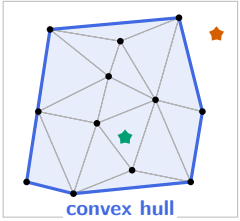
Spline was worst here but best for $\sin x$. **Safe practice:** go only a short distance, compare methods, let `nan` warn you, and **always plot**.

2-D: grid or scattered?

Data on a grid



Scattered data



Tensor product: 1-D along x, then y

- **Use when** data sit on a regular grid: images, tables, simulations
- `RegularGridInterpolator`
- `"cubic"` can overshoot; `"pchip"` won't

Triangles inside, nothing outside

Convex hull = smallest convex region holding all points.

★ inside: `griddata` interpolates on its triangle.

★ outside: `griddata` gives `nan`.

`RBFInterpolator`: smooth everywhere ⇒ silently **extrapolates**.

Take-home

Interpolation

- Linear: local, safe, rough.
- Piecewise cubics: methods differ only in the slopes d_i.
- Spline = smooth & accurate; PCHIP = keeps the shape.
- 2-D: grid $\rightarrow$ tensor product; scattered $\rightarrow$ triangles or RBF.

Extrapolation

- Each method just continues its end formula; they disagree quickly.
- Guarantees (shape, accuracy) hold only inside the data.
- `nan` outside is a *feature*: it warns you.
- Short distances only, and always plot.

Inside the data you choose a method; outside, you are guessing.

Beyond: add statistics ⇒ Gaussian process interpolation

So far: one curve. With statistics: a curve **and** how much to trust it.

Idea: a curve with error bars

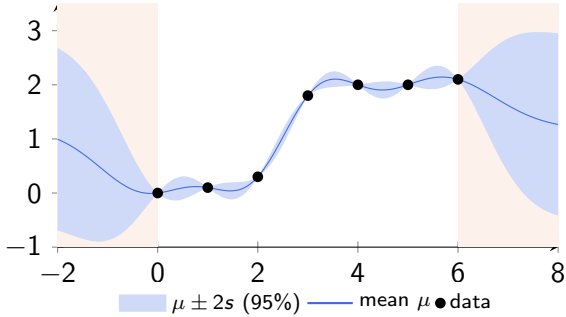
Treat f as **random**; nearby x give similar values:

$$\operatorname{cov}\big(f(x), f(x')\big) = \sigma^2 e^{-(x-x')^2/2\ell^2}$$

Condition on the data ⇒ **mean** $\mu(x)$ = the interpolant (through the points, like RBF) and **std** $s(x)$ = how unsure we are.

Confidence window $\mu \pm 2s$

Zero at the data, small between points, **wide outside**: the method itself *shows* that extrapolation is a guess.



Far from the data: mean falls back to the average, band is wide.

```
GaussianProcessRegressor(kernel=RBF()).fit(X, y)
mu, s = gp.predict(Xnew, return_std=True)
```

(scikit-learn)